claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_471a9cdd-d1d\agent-afe22969736882cc6.jsonl`
- SHA-256: `9fea6e91ac4dc72819f9306189ea8612d756962dbd0bb17a72b994e24c04cc46`
- Source modified: `2026-06-10T09:33:15+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_471a9cdd-d1d`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T09:28:10.499Z)

CONTEXT ? you are advising a real architecture decision for a solo-founder project (owner = avidullu).
Repos on disk (read-only; cite file paths for every claim):
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? FastAPI+SQLite+ffmpeg rally/highlight pipeline. Pluggable registries (segmenters/providers/validators/sports/annotations). Key seams: backend/pipeline/segmenters/base.py (registry), backend/pipeline/detectors/base.py (DetectorRunner ABC ? WASB/TrackNet run as WSL subprocesses from an EXTERNAL repo ~/models/WASB-SBDT), backend/providers/ (Gemini = external API). Eval stack backend/eval/ (metrics.match_intervals = the scoring spine, rally_seq_proto.py = the learned Gen-0 prototype seed, eval_partial_labels, calibrate_local LOVO, regression gate w/ tracked baseline in eval_baselines/). CI = .github/workflows/ci.yml (an import-smoke job WITHOUT the GPU stack + a full-suite job with CPU torch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? separate repo, the data system-of-record (golden labels, licensing gate, CVAT ingestion, 'curate export' manifest consumed by the indexer's batch eval).
- Also exists: private sports-obsreport repo = a shared report library consumed as a SOFT dependency by both repos (a working precedent for 'separate repo + consumed as dep').

THE DECISION: the owner is about to greenlight M0.4 (productionize rally_seq_proto into a Gen-0 TAL train?eval?ship-gate harness over ActionFormer/ASFormer (MIT), later Gen-2 = Qwen3-VL fine-tuned via ms-swift) and M0.1 (corpus spine + event-index contract, authored in the collector). QUESTION: should the model/training work live IN the indexer repo, or in a SEPARATE repo whose output is consumed by the indexer as just another segmenter/runner (like WASB and Gemini)? The owner PREFERS isolation ("iterate independently", cites the collector split as precedent) but asked for honest thoughts.

KNOWN FACTS to weigh (verified in a deep audit this week ? do not re-litigate, build on them):
- Cross-repo contract drift IS a real observed cost here: the collector's export header (start,end) vs the indexer's calibrate_wasb loader (start_time,end_time) forked into two incompatible golden-CSV conventions; tuned constants (TUNED/BASELINE) were duplicated-by-copy across files and drifted.
- Import-graph hygiene is a live concern: importing backend.main pulls torch/cv2 via the segmenter registration manifest (lazy-imports were deferred to the async-job slice).
- Authoritative plan docs: docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md ('envision now, build later' ? design every deferred seam up front; train EXPORT-CLEAN ops; on-device later = exported ONNX/CoreML/GGUF artifact + thin shell; the moat = consented dataset + eval harness, NOT the weights), docs/NEXT_STEPS.md, docs/PLATFORM_ARCHITECTURE.md.
- Guardrails: MIT/Apache-2.0/BSD only for code+data+weights deps; trained weights themselves = private/proprietary asset; minors excluded from training; per-user training opt-in; consent ledger planned in M0.1 schema.
- Quality state: learned LOVO mean 0.626 (n=6) vs heuristic 0.480; ship-gate F1@tIoU target still undefined.

- A repo RENAME off 'badminton' is already an owner action item ? repo topology is in flux anyway.
- Training runs on the owner's Windows box via WSL2 + RTX 2070 Super (8GB); Gen-2 may use rented cloud GPUs (Modal/RunPod per the study).
- Solo developer + AI agents do the work; there are ALREADY 4-5 repos (indexer, collector, rally-annotator, sports-obsreport).
RULES: read-only. Your final structured output is consumed programmatically.
ROLE: argue the STRONGEST case for keeping the model/training work IN the indexer repo (e.g. a training/ or backend/models/ package), at least through Gen-0/Gen-1. Read first: backend/eval/ (how tightly rally_seq_proto + metrics + calibrate_local + eval_partial_labels interlock ? M0.4 explicitly says 'reusing metrics.match_intervals'), the audit-verified cross-repo drift facts above, docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md. Make the case concrete: how to get the owner's desired ISOLATION inside one repo (package boundaries, optional-dependency extras, separate CI jobs, CODEOWNERS-style discipline, no segmenter imports of training code), when the split SHOULD eventually happen (e.g. when ms-swift/Gen-2 arrives), and what 'split later' costs vs 'split now'. Be honest in 'risks' about import-graph creep and the temptation entropy of a monorepo.

2. user (2026-06-10T09:28:18.451Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/:
__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
export_wasb_segments.py
gemini_refine.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py

C:/Users/avidu/Projects/badminton-highlight-indexer/docs/:
BACKLOG.md
CODE_MAP.md
COMMERCIALIZATION.md
DEFERRED_HARDENING_PLAN.md
EVALUATION.md
GOLDEN_SET_IMPLEMENTATION.md
GOLDEN_SET_PHASE1_VIDEOS.md
GOLDEN_SET_VENDORING.md
HOW_RALLY_DETECTION_WORKS.md
NEXT_STEPS.md
OWNED_MODEL_IMPLEMENTATION_PLAN.md
PLATFORM_ARCHITECTURE.md
PMF_MARKET_RESEARCH.md
README.md
ROADMAP.md
TRACKNET_WSL_SETUP.md
VIDEO_RECORDING_GUIDELINES.md
archives

3. user (2026-06-10T09:28:19.093Z)

backend\eval__init__.py
backend\eval\annotations.py
backend\eval\classifier.py
backend\eval\training.py

```
backend\eval\calibration.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py
backend\eval\harness.py
backend\eval\batch.py
backend\eval\eval_partial_labels.py
backend\eval\metrics.py
backend\eval\rally_seq_proto.py
backend\eval\regression.py
```

4. user (2026-06-10T09:28:26.164Z)

```
1       """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
heuristic).
2
3       Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
4       rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
5       (per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in
6       the trajectory; the hand-tuned heuristic just can't extract it across footage. This
7       prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
8       recovers the rally segments ? including on the footage the heuristic fails on.
9
10      Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
11      LogisticRegression ? no torch/sklearn):
12        1. Per-frame features over a local window from the WASB trajectory: visibility rate,
13           mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
14           All are scale/camera-robust RATES (unlike a raw velocity threshold).
15        2. Per-frame label = inside a human GT rally or not.
16        3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
17           held-out one (the honest cross-footage number; no within-video leakage).
18        4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
19           -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
heuristic).
20
21      This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
22      hand features). It exists to answer one question: is the rally boundary learnable from
23      the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.
24
25      Run:
26          python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
27      """
28      from __future__ import annotations
29
30      import json
31      import argparse
32      from typing import Dict, List, Tuple
33
34      import numpy as np
35      from scipy.ndimage import uniform_filter1d, maximum_filter1d
36      from scipy.stats import rankdata
37
38      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
39      from backend.eval.classifier import LogisticRegression
40      from backend.eval.calibrate_wasb import load_golden_gts
41      from backend.eval.metrics import score
42
43      FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
44      Interval = Tuple[float, float]
45
46
```

```
    def _spread(a: np.ndarray) -> float:
        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0


    def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.

        Speed is normalized to fraction-of-frame-width PER SECOND (``* fps``), matching the
        heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
        speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
        on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
        occlusion gap (mirrors the heuristic resetting on invisibility)."""
        pts = sorted(points, key=lambda p: p.frame)
        N = (pts[-1].frame + 1) if pts else 1
        vis = np.zeros(N); x = np.zeros(N); y = np.zeros(N)
        for p in pts:
            if 0 <= p.frame < N and p.visible and p.x >= 0:
                vis[p.frame] = 1.0; x[p.frame] = float(p.x); y[p.frame] = float(p.y)

        mask = vis > 0
        play_is_x = _spread(x[mask]) >= _spread(y[mask])
        coord = x if play_is_x else y
        net = float(np.median(coord[mask])) if mask.any() else 0.0
        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

        speed = np.zeros(N); spd_mask = np.zeros(N); cross = np.zeros(N)
        max_gap = max(2, round(0.25 * fps))  # frames; a longer gap = occlusion break, not
motion
        prev = None
        for p in pts:
            f = p.frame
            if not (0 <= f < N and vis[f]):
                continue
            if prev is not None:
                df = f - prev
                if 0 < df <= max_gap:
                    speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
                    spd_mask[f] = 1.0
                    c, pc = coord[f] - net, coord[prev] - net
                    if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
                        cross[f] = 1.0
            prev = f
        return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)


    def features(arr: Dict, fps: float, win_sec: float = 0.75, stride_sec: float = 0.1):
        """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
        N = arr["N"]; fw = arr["fw"]
        size = max(3, int(win_sec * fps) * 2 + 1)
        vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))

        n = uniform_filter1d(vis, size, mode="constant")                      # mean
visibility (== vis_rate)
        sm = uniform_filter1d(spd_mask, size, mode="constant")
        spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
        spd_max = maximum_filter1d(speed, size, mode="constant")
        cross_rate = uniform_filter1d(cross, size, mode="constant") * fps       # crossings
/ second
```

```
        xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
        x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
        x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
        ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
        y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
        y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw

        idx = np.arange(0, N, max(1, int(stride_sec * fps)))
        X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
        return np.nan_to_num(X), idx / fps


    def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
        y = np.zeros(len(times))
        for i, t in enumerate(times):
            for s, e in gts:
                if s <= t <= e:
                    y[i] = 1.0
                    break
        return y


    def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
        """Rank-based AUC (Mann-Whitney) with MIDRANK tie handling.

        Uses scipy.stats.rankdata (average method) so tied probabilities get averaged ranks
?
        without this, an all-tied classifier scored a spurious order-dependent 0.25..0.75
        instead of 0.5, and dead-time frames produce identical all-zero feature rows
(guaranteed
        ties), so the headline AUC carried an unquantified bias."""
        pos, neg = p[y > 0.5], p[y <= 0.5]
        if pos.size == 0 or neg.size == 0:
            return float("nan")
        ranks = rankdata(p)  # midranks (ties averaged)
        return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
neg.size))


    def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,
                          merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
List[Interval]:
        p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
        on = p >= threshold
        runs: List[Interval] = []
        i, n = 0, len(on)
        while i < n:
            if on[i]:
                j = i
                while j + 1 < n and on[j + 1]:
                    j += 1
                runs.append((float(times[i]), float(times[j])))
                i = j + 1
            else:
                i += 1
        merged: List[Interval] = []
        for s, e in runs:
            if merged and s - merged[-1][1] <= merge_gap:
                merged[-1] = (merged[-1][0], e)
            else:
                merged.append((s, e))
        return [(s, e) for s, e in merged if e - s >= min_dur]


```

```
    def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
        f1s = []
        for v in vids:
            segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
            f1s.append(score(segs, v["gts"], 0.5).f1)
        return sum(f1s) / max(len(f1s), 1)


    def run(specs: List[Dict]) -> dict:
        data = {}
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
            X, times = features(arr, float(s["fps"]))
            gts = load_golden_gts(s["labels"])
            data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
        names = list(data)
        print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
        print(f"features: {FEAT_NAMES}\n")

        results = []
        for test in names:
            train = [data[n] for n in names if n != test]
            Xtr = np.vstack([v["X"] for v in train]); ytr = np.concatenate([v["y"] for v in
train])
            clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
            clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
            # tune the decision threshold on TRAIN segment-F1 (no test leakage)
            thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
            v = data[test]
            probs = clf.predict_proba(v["X"])
            auc = roc_auc(v["y"], probs)
            segs = probs_to_segments(probs, v["times"], float(thr))
            sc = score(segs, v["gts"], 0.5)
            # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
            # from the threshold-transfer gap. Optimistic ? for diagnosis only.
            othr = max(np.arange(0.2, 0.81, 0.05),
                       key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
            osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
            results.append({"video": test, "thr": float(thr), "auc": auc,
                            "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
                            "n_pred": len(segs), "n_gt": len(v["gts"]),
                            "oracle_thr": float(othr), "oracle_f1": osc.f1})
            print(f"[held-out {test}]  per-frame AUC {auc:.3f}  |  LOVO-threshold F1
{sc.f1:.3f} "
                  f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train)  |  "
                  f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
        mean_f1 = sum(r["f1"] for r in results) / len(results)
        print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
        return {"per_video": results, "mean_f1": mean_f1}


    def main() -> None:
        ap = argparse.ArgumentParser(description=__doc__)
        ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
        args = ap.parse_args()
        with open(args.manifest, encoding="utf-8") as f:
            run(json.load(f))
```

```
218
219
220     if __name__ == "__main__":
221         main()
222
```

5. user (2026-06-10T09:28:26.540Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
24              return 0.0
25          inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26          if inter <= 0.0:
27              return 0.0
28          union = (a_end - a_start) + (b_end - b_start) - inter
29          return inter / union if union > 0 else 0.0
30
31
32      @dataclass
33      class Match:
34          """One matched predicted?ground-truth pair."""
35          pred_index: int
36          gt_index: int
37          iou: float
38
39
40      @dataclass
41      class MatchResult:
42          """Outcome of matching predictions to ground truth at a given IoU threshold."""
43          matches: List[Match] = field(default_factory=list)
44          unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
45          unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)
46
47
48      def match_intervals(preds: List[Interval], gts: List[Interval],
49                          iou_threshold: float = 0.5) -> MatchResult:
50          """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52          Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53          are claimed highest-IoU first, and each prediction/GT can be used once. This
54          keeps a single prediction from being credited for two ground-truth rallies
55          (and vice versa).
56          """
```

```
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
                # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
                if iou > 0.0 and iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
        false_negatives: int
        precision: float
        recall: float
        f1: float
        mean_iou: float            # over matched pairs (0.0 if none)
        mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
        mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

        def as_dict(self) -> dict:
            return self.__dict__.copy()


    def score(preds: List[Interval], gts: List[Interval],
              iou_threshold: float = 0.5,
              match_result: Optional[MatchResult] = None) -> Score:
        """Compute precision/recall/F1, mean matched IoU, and boundary errors.

        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
        extracting the failure lists); otherwise matching is done here.
        """
        mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

        tp = len(mr.matches)
        fp = len(mr.unmatched_pred_indices)
        fn = len(mr.unmatched_gt_indices)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
```

```
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
121
122         if mr.matches:
123             mean_iou = sum(m.iou for m in mr.matches) / tp
124             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
125             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
126         else:
127             mean_iou = mean_start_err = mean_end_err = 0.0
128
129         return Score(
130             iou_threshold=iou_threshold,
131             num_pred=len(preds), num_gt=len(gts),
132             true_positives=tp, false_positives=fp, false_negatives=fn,
133             precision=precision, recall=recall, f1=f1,
134             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
135         )
136
137
138     def pr_curve(preds: List[Interval], gts: List[Interval],
139                  thresholds: Optional[List[float]] = None) -> List[Score]:
140         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

6. user (2026-06-10T09:28:31.619Z)

```
1       # Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3       **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner
greenlights a specific
4       milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
training source).
5       **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO +
learned prototype
6       (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy
workflow (2026-06-09).
7
8       ## Design principle (owner directive): *envision now, build later*
9       Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be
**architecturally envisioned
10      now** so we never hit a mammoth refactor. We **design the seams** (data contracts,
export boundaries, the
11      event store) up front and **implement behind them** when each milestone is greenlit.
Punting a *feature* is fine;
12      punting its *seam* is not.
13
14      ## North Star (reframed per PR #61 review)
15      - **Immediate deliverable:** a **training framework + harness** that produces a model
with **very high precision/
16        recall for extracting highlights + rallies from a clip**. *Quality is the first-order
objective.*
17      - **Eventual product:** a **single, sport-agnostic, conversational** video model ?
upload a clip, then ask
18        contextual questions ("longest rally", "make a clip of all service faults", "rallies
over 20 shots").
19      - **On-device is LATER** (a model-compression goal so power users save upload
cost/hassle) ? definitely on the
```

```
20        roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21      - **The moat = the consented dataset + the eval harness, not the weights.**
22
23      ## Hard guardrails (non-negotiable ? "never corrupted")
24      - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT
/ Apache-2.0 / BSD only.**
25        (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-
restrictive: Gemma 3 (viral),
26        **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU
cap), Commons-Clause repos.
27      - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public
grounding-CoT datasets that
28        were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision
(a real, binding cost).
29      - **Exclude minors; per-user training data needs a separate explicit opt-in; split data
by match.**
30      - ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl.
Qwen3-VL).** Our "no paid-LLM
31        training" rule is enforceable at *our fine-tuning data* layer, not the base's
pretraining ? an **explicit owner
32        risk-acceptance** is required to use any open VLM. (Governed by the base's license,
not our pipeline.)
33
34      ## The model strategy (verified 2026-06-09)
35      **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge
(runs SFT/GRPO, emits
36      weights); a *base* is the metal you forge. You pick one framework and feed it any clean
base.
37      - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio
+ timestamp-grounding +
38        GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core;
*avoid its AGPL Studio UI*)
39        as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-
time snapshot ? re-verify at impl.
40      - **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-
video context, purpose-built
41        **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm
the exact size's LICENSE
42        pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but
its Prohibited-Use posture is
43        unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not
the primary extractor.
44        **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is
cheap *only within
45        timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-
specific).
46      - **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss
strict temporal boundaries**
47        (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse
regions, not accurate boundaries").
48        **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-
risked on our corpus (held-out
49        AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params
(trivially compressible later),
50        and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So:
**head now ? VLM later**,
51        strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes,
head refines boundaries) over
52        wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur
footage may differ ? re-measure.)*
53
54      ## Conversational video-QA ? build the bookkeeping NOW, punt the model
55      **Punt the conversational *feature*** until extraction quality is solid (current LOVO
~0.583 ? "very high"; a chat UI
56      over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn**
```

(expensive; bad at exact
57   counting; the strong teachers are license-forbidden). **But build the *seam* now** (the
brief requires it):
58   - **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits
structured timestamped
59   spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-
SQL + ffmpeg clip-cut). The
60   cited queries ("longest rally", "all service faults", "rallies > 20 shots") are
**structured-numeric ? SQL, not a
61   VLM job** (SQL counts exactly where VLMs hallucinate).
62   - **Designed-now seam:** make the **per-video event-index schema a first-class data
contract** alongside the M0.1
63   corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count,
ending_reason/fault_tags,
64   derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy
(e.g. `service_fault`) up front**
65   even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration):
`play_segments` today has
66   `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights`
isn't a table yet ? so it's an
67   *extension with new columns + a highlights table*, not a no-op. **Punt** the NL
parser/agent until quality clears the bar.
68   - Paid Gemini may answer genuinely open-ended edge questions **at inference** over our
structured index (allowed ?
69   it reasons over our data, it is not trained on its outputs).
70
71   ## Multisport (single sport-agnostic model)
72   - **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned
across sports (sport = an input
73   feature / per-sport calibration). The collector + indexer are already sport-generic. ?
**ACTION: rename the repo off
74   "badminton" ASAP** (owner emphatic) ? consistent with this.
75   - **What multisport changes = DATA + DETECTORS (scale up, the accepted "good
problem"):** each new sport needs its own
76   labeled matches (split by match) + a **clean trajectory detector** to feed the head's
features. **The binding blocker
77   is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-
provenance issue, and **clean MIT/
78   Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified.** Sequence by public-data
79   richness: **badminton ? tennis/TT ? pickleball/padel.**
80   - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81   and the evidence once cited for it (RacketVision) was *refuted on verification* (it's
ball-tracking, not temporal). The
82   concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83   corpus exists. Don't cite RacketVision as evidence.
84
85   ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86   - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87   - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-
process (ffmpeg, WASB-in-WSL, cloud
88   GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89   ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90   - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language:** train in
91   Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92   runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression

```
93        target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94      - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95        separate service. **None exists today.**
96
97      ---
98
99      ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) ·
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) ·
*greenlight item*
115     - **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116       **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV
labels ? `output/local_rally_model.json`)
117       and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118       source-agnostic* function ? there is no store by that name; the earlier
"training.label_candidates" pointer was wrong.)
119       Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one
120       command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini =
eval/reference/fallback only.
121     - **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
122       blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean MIT/Apache detector swap.
123       ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our
temporal labels can't train the
124       shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies
per sport** (incl. any TT-on-WASB use).
125       Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
126
127     ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
128     Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
129     rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
130     reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
131     heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
```

```
132     F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
133
134     ---
135
136     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
137     Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
138     extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
139
140     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
141     - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe
142       (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
143       parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
144     - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
145       our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
146     - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) **+ a real
147       VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 6?8 GPUs ? **budget is the
148       largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
149
150     ---
151
152     ## Decisions / status
153     | # | item | status |
154     |---|---|---|
155     | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
156     | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157     | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158     | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159     | 5 | Rename repo off "badminton" | **owner action** ? |
160     | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161
162     ## Open risks / unknowns (none block Phase-0 research)
163     Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
164     trajectory detectors not yet identified · base-model pretraining provenance unauditable
(risk-acceptance) · real VLM
165     VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
166     training · on-device 4B latency on 8 GB unmeasured.
167
168     ---
169     *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170     next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
171
```

7. user (2026-06-10T09:28:39.409Z)

```
1       """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2       on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3       100% local (no cloud, privacy lever intact, ~zero marginal cost).
```

```
4
5       Pipeline being tuned (all local, no Gemini at runtime):
6           WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                                -> rally_gate (min_crossings)            -> rally windows
8
9       Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10      videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11      the honest "how precise is the local pipeline on an unseen video" number. The
12      output is a single local config + the precision it buys, with NO Gemini dependency
13      shipped.
14
15      Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16      calibration.{select_best, leave_one_video_out, improvement_report}.
17
18      Run:
19          python -m backend.eval.calibrate_local --manifest videos.json
20      where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21      (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Callable, Dict, List, Tuple
28
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30      from backend.pipeline.detectors import rally_gate
31      from backend.eval.calibrate_wasb import load_golden_gts
32      from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34      # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35      LocalConfig = Tuple[float, float, float, int]
36      BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0)   # today's local default (windowing,
no gate)
37
38
39      def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40          """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41          (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
42          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
43
44          def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
45              vt, mg, mwd, mc = cfg
46              wins = TrackNetRunner.trajectory_to_action_windows(
47                  points, fps=fps, frame_width=frame_width,
48                  velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
49              )
50              wins = [(w["start"], w["end"]) for w in wins]
51              if mc > 0:
52                  wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
53              return wins
54
55          return predict_fn
56
57
58      def local_grid() -> List[LocalConfig]:
59          return [(vt, mg, mwd, mc)
60                  for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
61                  for mg in (1.0, 2.0, 3.0)
62                  for mwd in (1.0, 2.0, 3.0)
63                  for mc in (0, 1, 2, 3)]
64
65
```

```
66      def build_videos(specs: List[Dict]) -> List[Dict]:
67          videos = []
68          for s in specs:
69              pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
float(s["frame_width"]))
70              gts = load_golden_gts(s["labels"])
71              if not gts:
72                  raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
73              videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
gts})
74          return videos
75
76
77      def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
78          videos = build_videos(specs)
79          grid = local_grid()
80          print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
```

8. user (2026-06-10T09:28:40.350Z)

```
1       """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3       The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4       ending_reason,sport). While a video is only partially tagged, the labels cover a
5       PREFIX of the video ? every rally up to some point is marked, and everything after is
6       simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7       the detector's *correct* rallies in the un-labeled tail would be counted as false
8       positives and crush precision.
9
10      This harness fixes that by restricting the evaluation to the LABELED WINDOW
11      [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12      dropping predicted segments that fall outside it. Within that window the human labels
13      ARE complete, so precision/recall/F1 are honest.
14
15      It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16        - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17        - pipeline.detectors.rally_gate.apply_gate   (net-crossing "gate A")
18        - eval.metrics.match_intervals / score
19        - export_wasb_segments.build_comparison / write_comparison_csv  (per-rally detail CSV)
20
21      Run:
22          python -m backend.eval.eval_partial_labels \
23              --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24              --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
Avi.rallies.csv" \
25              --fps 59.94 --frame-width 1920
26      """
27      from __future__ import annotations
28
29      import argparse
30      import os.path as osp
31      from typing import List, Optional, Tuple
32
33      from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
34      from backend.eval.metrics import score
35      from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
36      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37      from backend.pipeline.detectors import rally_gate
38
39      Interval = Tuple[float, float]
40      Config = Tuple[float, float, float]
41
42      # Operating points to compare. The tuned config wins on its fit-video but over-segments
43      # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44      TUNED: Config = (0.05, 1.0, 1.0)
```

```

    def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
        """Keep intervals that overlap the labeled window [w0, w1].

        A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
        in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
        they are neither rewarded nor penalized. GTs are assumed already inside the window.
        """
        return [(s, e) for (s, e) in intervals if e > w0 and s < w1]


    def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
                    window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
        """Score one (config, gate) operating point inside the labeled window."""
        preds = predict_fn(cfg)
        if gate_crossings > 0 and points:
            preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
        preds = restrict_to_window(preds, *window)
        sc = score(preds, gts, iou)
        return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
"preds": preds}


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            window_start: float = 0.0, window_end: Optional[float] = None,
            gate_crossings: int = 2, iou: float = 0.5,
```

9. user (2026-06-10T09:28:40.593Z)

```
    name: CI

    on:
      push:
        branches: [master]
      pull_request:

    permissions:
      contents: read

    concurrency:
      group: ci-${{ github.ref }}
      cancel-in-progress: true

    jobs:
      # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
      # module-level import error shipping while the suite stays green.
      # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
      # on machines without the GPU stack (test_import_smoke stubs what's absent).
      import-smoke:
        name: Syntax sweep + import smoke (no GPU stack)
        runs-on: ubuntu-latest
        steps:
          - uses: actions/checkout@v4
          - uses: actions/setup-python@v5
            with:
              python-version: "3.13"
          - name: Install minimal deps
            run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
          - name: Run import smoke tests
            run: python -m pytest tests/test_import_smoke.py -v

      full-suite:
```

```
34          name: Full test suite (offline, mocked)
35          runs-on: ubuntu-latest
36          steps:
37            - uses: actions/checkout@v4
38            - uses: actions/setup-python@v5
39              with:
40                python-version: "3.13"
41                cache: pip
42            - name: System libs for opencv-python
43              run: |
44                sudo apt-get update
45                sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
46            - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
47              run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
48            - name: Install requirements
49              run: python -m pip install -r requirements.txt pytest httpx  # httpx: fastapi
TestClient
50            # obsreport (private, soft dep) is absent here: reporting tests skip via
51            # pytest.importorskip ? run_all_tests.py prints which.
52            - name: Run full suite
53              run: python run_all_tests.py
54
```

10. user (2026-06-10T09:28:50.355Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import hashlib
20      import logging
21      from dataclasses import dataclass, asdict
22      from typing import Dict, List, Optional
23
24      from backend.infrastructure.database import Database
25      from backend.eval import metrics
26      from backend.eval.harness import get_predictions
27      from backend.annotations import annotation_registry
28
29      logger = logging.getLogger(__name__)
30
31      # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32      # actually load a committed baseline to compare against.
33      DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34      DEFAULT_TOLERANCE = 0.05
35
36
37      def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38          """Stable short hash of the GT intervals ? detects a baseline taken against
```

```
different labels."""
39          norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40          return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43      @dataclass
44      class RegressionResult:
45          video_id: str
46          stage: str
47          iou_threshold: float
48          precision: float
49          recall: float
50          f1: float
51          # Baseline values compared against (None when no baseline existed yet).
52          baseline_precision: Optional[float]
53          baseline_recall: Optional[float]
54          tolerance: float
55          regressed: bool
56          reasons: List[str]
57          # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58          # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59          has_baseline: bool = False
60          gt_hash: Optional[str] = None
```

11. user (2026-06-10T09:28:54.398Z)

```
No files found
```

12. user (2026-06-10T09:28:54.680Z)

```
1       # Next steps (live) ? owned-model + multisport
2
3       **Updated:** 2026-06-10 (post audit-hardening sweep). **Authoritative roadmap:**
4       `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`. **Latest blow-by-blow:** memory `current-
state.md` / `session-handoff.md`.
5       This file = the prioritized "what to do next," refreshed each session.
6
7       > **2026-06-10 sweep (12 PRs, both repos, all merged):** CI now exists in both repos;
master un-broken (compiler
8       > SyntaxError fix); detector artifacts keyed by video_id; re-ingest is destroy-after-
confirm; `indexing.wasb` config
9       > real; eval gate hardened (tracked baseline, exit-3 on missing, GT-hash); API path
validation; WSL/provider timeouts +
10      > SQLite WAL; collector licensing gate + golden-label import guard + CVAT gate fixes;
**corpus reclassified** (23
11      > ShuttleSet broadcast rows demoted to non-commercial ? the commercial export is now
owned-data-only); docs truth-synced.
12      > ? **Metric corrections re-validated the quality story and strengthened it: learned
LOVO 0.583 ? 0.626 vs heuristic
13      > 0.480** (midrank AUC + fps-normalized speed; per-video AUC unchanged 0.83?0.92).
14
15      ## The recommendation (one paragraph)
16      Build the **quality-first extraction harness** before anything else: a learned **TAL
head** (ActionFormer/ASFormer, MIT)
17      over trajectory features is the robust path ? it **decouples from the multi-court
"background games" confound** that the
18      unsupervised heuristic (and audio, and spatial court-gating) can't escape (all three
tested + failed; it's a *temporal*
19      rally-vs-non-rally problem). **Framework = ms-swift, base = Qwen3-VL** (Gemma-4
AMBER/bench) come **later** for the
20      conversational/grounding upgrade ? start with the head. **On-device is deferred**
(export-clean ops now so it stays
21      reachable). Everything stays **MIT/Apache-2.0/BSD** (code + data + weights). Bottleneck
= **data + per-sport trajectory
```

```
22      detectors**, not model architecture.
23
24      ## Latest findings (2026-06-09)
25      - **6-match badminton LOVO:** learned prototype **0.583** vs heuristic **0.480**
(learned leads since n=3, AUC 0.83?0.92,
26        decoupled from multi-court). The "contrast" metric (in-rally÷dead-time visibility)
predicts heuristic F1 monotonically.
27      - **?? Multisport (Roora pickleball + TT):** labels **ingest cleanly** (`curate convert-
cvat` ? 6 rallies/sport, sane).
28        **WASB-transfer SURPRISE (directional, n=6 each ? TINY):** the badminton shuttle
detector segments **TABLE TENNIS at
29        F1 0.909** out-of-the-box (!) but **pickleball only 0.308**. ? **The per-sport-
detector blocker is SPORT-DEPENDENT:** TT may
30        be servable with WASB short-term; pickleball needs a proper (MIT/Apache) ball
detector. Confirm/deny with more videos.
31      - **Owner field notes (2026-06-09):** camera height/distance **varied on purpose**
(great generalization data); **multi-court
32        adjacent-court SOUND dominates the signal** ? empirically reconfirms audio-foreground
isolation won't work + the
33        robustness-first learned-model path. Multi-court is the realistic (academy)
environment, so this *is* the target distribution.
34
35      ## Prioritized next steps
36      1. **[owner greenlight] Phase-0 (M0.x), $0/local:**
37         - **M0.4 ? Gen-0 TAL harness** (first-class): productionize `rally_seq_proto.py` ?
one-command train?eval?**ship-gate** over
38           ActionFormer/ASFormer, reusing `metrics.match_intervals`. **Define the ship-gate
target** (explicit F1@tIoU; floor = beat
39           heuristic LOVO **0.480**; the corrected learned prototype already sits at
**0.626**) ? "very high P/R" must be
40           operationalized, not just "beat the floor." Add **paired per-video stats**
(sign/Wilcoxon) ? at n=6 a mean gap alone is noise-ambiguous.
41         - **M0.1 ? corpus spine + the per-video event-index contract** (the conversational
seam: extract ? event store ? text-to-SQL).
42           Includes importing the n=6 golden matches into the collector (now safe: import
guard + `--normalize`, PR #15).
43         - **M0.3 ? compliance:** ? corpus licensing reclass DONE (collector #14/#17 ?
ShuttleSet demoted, export owned-only).
44           REMAINING: retire/retarget the Gemini-silver training path
(`backend/eval/distill_local.py` + `calibrate_local`
45           thresholds; artifacts in gitignored `output/`); WASB **C3** own-weights/clean-swap
(gates ship; un-scoped new
46           workstream ? temporal labels can't train the detector, per ADR-002).
47      2. **Multisport thread:**
48         - Ingest the Roora pickleball/TT rally CSVs into the corpus (collector `import-local
--normalize`).
49         - **Get more pickleball + TT videos** ? re-run the WASB-transfer eval to confirm/deny
TT?0.91, pickleball?0.31 (n=6 is too few).
50         - Once each sport has ?3 matches, run a **merged prototype LOVO across badminton + TT
(+ pickleball)** ? the "how would a
51           merged model do" experiment. Identify a clean MIT/Apache ball detector for sports
where WASB doesn't transfer (pickleball).
52           ? Any TT-on-WASB serving is still gated by C3 (checkpoint provenance multiplies per
sport).
53      3. **PMF validation (cheaper than any engineering month ? audit finding: the "India
badminton whitespace" was falsified;
54         Bangalore has ?4 funded players; the surviving wedge = software-only multi-court BYO-
footage indexing):**
55         - **C13 discovery interviews** (10?15 Bengaluru academies) + two added questions: "do
you have multi-court footage
56           nobody watches?" and "would you pay per-match to have it indexed?"
57         - **Gemini-wrapper baseline vs the n=6 LOVO corpus** (harness exists) ? if a wrapper
? learned 0.626, the owned-model
58           investment is premature and the play is distribution/UX.
59      4. **Housekeeping:** **rename repo off "badminton"** (multisport, owner action ? now in
```

```
BACKLOG); confirm collector =
60         corpus/event-store system-of-record.
61
62      ## Pointers / artifacts
63      - Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Study:
`docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`
64        (2026-06-09 banner) · Quality: `docs/archives/quality-iterations/2026-06-multivideo-
lovo/`.
65      - Eval corpus: `output/human_lovo_manifest.json` (badminton n=6; gitignored ? back it up
with the artifacts). **Durable
66        artifacts: `C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\`** (all
WASB trajectories + `roora\`
67        pickleball/TT CSVs+XMLs ? rescued from `%TEMP%`). Labels: `Annotation
Setup\Collect\Golden Labelled\`.
68        Code: `backend/eval/{eval_partial_labels,rally_seq_proto,calibrate_local}.py`.
69      - Collector corpus backup (pre-reclass): `sports-data-
collector/data/sports_metadata.backup-2026-06-10.db` (gitignored).
70      - ? No owned-model implementation until owner greenlights a milestone. Committed next
PLATFORM slice = async job model (single-tenant).
71
```

13. user (2026-06-10T09:29:06.295Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
```

```
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

14. user (2026-06-10T09:29:07.774Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/:
CLAUDE.md
README.md
TEST_RUN_SUMMARY.md
__pycache__
backend
config.json
docs
output
pytest.ini
requirements.txt
run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py
scratch
scripts
setup.py
tests

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/:
__init__.py
__pycache__
base.py
rally_gate.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/:
__init__.py
__pycache__
base.py
gemini.py
motion.py
tracknet_hybrid.py
```

```
wasb_hybrid.py
yolo_hybrid.py
```

15. user (2026-06-10T09:29:23.550Z)

```
generate_khelacharya_flyer.py
```

16. user (2026-06-10T09:29:23.556Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4       from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7
8       __all__ = [
9           "segmenter_registry",
10          "BasePlaySegmenter",
11          "MotionPlaySegmenter",
12          "GeminiPlaySegmenter",
13          "HybridYoloSegmenter",
14          "TrackNetHybridSegmenter",
15          "WasbHybridSegmenter",
16      ]
17
```

17. user (2026-06-10T09:29:23.557Z)

```
1       """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3       This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4       of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5       the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7       The ABC lets us:
8       - Keep the current WSL adapters working unchanged for Windows users.
9       - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11      - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14      Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15      - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16        Run detector, return the Windows-side path to the produced trajectory CSV
17        (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18        The caller (hybrid) is responsible for output dir; impls may stage the video
19        into WSL fs for perf.
20
21      - healthcheck() -> tuple[bool, str]
22        Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23        Message is logged (and can be surfaced to reports or UX in future).
24        Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26      Existing behavior is 100% preserved. The only additions are the inheritance and
27      the explicit common type for future implementers.
28
29      How to add a new runner (for docs / future contributors):
30      1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31      2. Implement run_predict and healthcheck (return (bool, str) tuple).
32      3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33         TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34         deliberately static and model-agnostic). If the new detector needs different
35         windowing, it can implement its own or we generalize later.
```

```
    4. Lazy-import heavy native deps inside the methods (so importing the segmenter
       registry never pulls torch/tf on a machine that doesn't have them).
    5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
       DetectorRunner instances (they already do the right thing via duck-typing + the
       type hints we add here).
    6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
    7. No change to eval/calibration code paths (they consume the CSV + the static
       windowing helpers directly).

    Optional Linux-native path (the main de-risk goal):
    A LinuxNativeRunner (or ONNXRunner) would:
    - Skip all wsl / _stage / conda activate.
    - Load weights locally (torch or onnx) or call a native binary.
    - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
    - healthcheck can just check "weights file exists and torch importable + CUDA?".
    This removes the WSL requirement for Linux users / future SaaS workers while
    keeping the exact same candidate quality characteristics (or better, once native
    weights are available).

    Error surfacing into reports:
    The healthcheck message is already logged by the hybrids on failure. The reporting
    harvest sees the outcome as "0 candidates" + detector name. When we wire richer
    segmenter Results (or pass detector_health into emit_indexer_report context), the
    (ok, msg) can be recorded explicitly under segmentation.details or a new detector
    stage. The ABC makes that future change local to one place.

    See also:
    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
    """

    from abc import ABC, abstractmethod
    from typing import Optional, Tuple


    class DetectorRunner(ABC):
        """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""

        @abstractmethod
        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run inference and return path to trajectory CSV (or None on failure).

            ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
            videos that happen to share a filename cannot reuse each other's frames or CSV.
            It is optional for back-compat; callers in the live pipeline always pass it.
            """
            pass

        @abstractmethod
        def healthcheck(self) -> Tuple[bool, str]:
            """Quick check if the runner (env, weights, etc.) is usable.

            Returns (ok, message). Never raises. Cheap to call before a long run.
            """
            pass
```

18. user (2026-06-10T09:29:35.808Z)

```
    """Batch evaluation across a collector export manifest.

    Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
```

```
4       sports-data-collector's `curate export`, (optionally) process each video through
5       a segmenter so detections land in the indexer DB, then score every video's
6       detected rallies against its ground-truth CSV and aggregate the results.
7
8       This module holds the *pure* pieces (path resolution, stage selection, metric
9       aggregation) so they can be unit-tested without touching video files, the GPU,
10      or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11      """
12
13      import os
14      from typing import Dict, List, Optional
15
16
17      # Segmenters that only ever populate the `final` play_segments table (no raw
18      # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19      _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22      def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23          """Resolve a manifest `local_path` to an absolute path.
24
25          The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
26          to its own repo root and with Windows separators. Normalise separators, strip
27          a leading ``./``, and resolve relative paths against `videos_root` (the
28          collector root). Returns None for a missing/empty path.
29          """
30          if not local_path:
31              return None
32          p = local_path.replace("\\", "/")
33          if p.startswith("./"):
34              p = p[2:]
35          # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36          # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37          # This keeps collector manifest paths working in tests and mixed environments.
38          if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39              return os.path.normpath(p)
40          return os.path.normpath(os.path.join(videos_root, p))
41
42
43      def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44          """Pick which prediction stages to score.
45
46          ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47          but only ``final`` for segmenters that don't emit candidates. An explicit
48          request ('candidates' | 'final' | 'both') overrides the heuristic.
49          """
50          if requested == "both":
51              return ["candidates", "final"]
52          if requested in ("candidates", "final"):
53              return [requested]
54          # auto
55          if segmenter in _FINAL_ONLY_SEGMENTERS:
56              return ["final"]
57          return ["candidates", "final"]
58
59
60      def _safe_div(num: float, den: float) -> float:
```

19. user (2026-06-10T09:29:36.841Z)

```
1       """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3       Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
```

```
        -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
        -> backend.eval.calibration (improvement_report / cross_validate)

    Run (after a full-video WASB pass exists):
        python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
            --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920

    Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
    same GT is an **optimistic** upper bound. The trustworthy number is the
    cross-validated held-out **recall** (and, once a 2nd labelled video exists,
    `leave_one_video_out` for precision/F1 generalization).
    """
    from __future__ import annotations

    import csv
    import argparse
    from typing import List, Tuple, Callable

    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.eval.calibration import (
        select_best, improvement_report, cross_validate, evaluate,
    )

    Interval = Tuple[float, float]
    Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)

    BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults


    def load_golden_gts(csv_path: str) -> List[Interval]:
        """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
        gts: List[Interval] = []
        with open(csv_path, newline="", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
            for row in reader:
                row = {(k or "").strip().lower(): v for k, v in row.items()}
                try:
                    s = float((row.get("start_time") or "").strip())
                    e = float((row.get("end_time") or "").strip())
                except (TypeError, ValueError):
                    continue
                if e > s:
                    gts.append((s, e))
        return sorted(gts)


    def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: Config) -> List[Interval]:
            vt, mg, mwd = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            return [(w["start"], w["end"]) for w in wins]

        return predict_fn

```

```
66
67      def default_grid() -> List[Config]:
68          return [(vt, mg, mwd)
69                  for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
70                  for mg in (1.0, 2.0, 3.0)
71                  for mwd in (1.0, 2.0, 3.0)]
72
73
74      def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
75              iou: float = 0.5) -> dict:
76          gts = load_golden_gts(labels_csv)
77          if not gts:
78              raise SystemExit(f"no usable golden rallies in {labels_csv}")
79          predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
80          grid = default_grid()
81
82          base = evaluate(predict_fn(BASELINE), gts, iou)
83          best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
84          fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
85          cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall
86
87          print("=== WASB rally-segmentation calibration ===")
88          print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
89          print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
90                f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
```

20. user (2026-06-10T09:29:37.758Z)

```
.github\workflows\ci.yml:50:      # obsreport (private, soft dep) is absent here: reporting
tests skip via
.github\workflows\ci.yml:51:      # pytest.importorskip ? run_all_tests.py prints which.
backend\reporting\__init__.py:3:`obsreport` is a SOFT dependency: if it is not installed, this
module degrades to a
backend\reporting\__init__.py:14:    import obsreport  # noqa: F401
backend\reporting\__init__.py:18:except Exception:  # pragma: no cover - exercised only when
obsreport is absent
backend\reporting\harvest.py:4:and run metadata, populate an `obsreport.RunRecorder` with the
raw signals. The
backend\reporting\harvest.py:8:Imported only after `backend.reporting` confirms obsreport is
installed, so it may
backend\reporting\harvest.py:9:import obsreport freely.
backend\reporting\harvest.py:18:    import obsreport  # soft dep; only needed for RunRecorder /
load_spec at runtime
backend\reporting\harvest.py:20:    obsreport = None  # type: ignore[assignment]
backend\reporting\harvest.py:26:def load_spec() -> "obsreport.ReportSpec":
backend\reporting\harvest.py:27:    return obsreport.load_spec(_SPEC_PATH)
backend\reporting\harvest.py:80:def _validation_stage(rec: "obsreport.RunRecorder", val_results:
List[Any]) -> None:
backend\reporting\harvest.py:93:    rec: "obsreport.RunRecorder",
backend\reporting\harvest.py:137:def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str,
Any], segmenter_actual: Optional[str],
backend\reporting\harvest.py:157:def _highlights(rec: "obsreport.RunRecorder", play_segments:
List[Dict[str, Any]],
backend\reporting\harvest.py:186:    spec: Optional["obsreport.ReportSpec"] = None,
backend\reporting\harvest.py:188:) -> "obsreport.RunRecorder":
backend\reporting\harvest.py:190:    rec = obsreport.RunRecorder(
backend\reporting\spec.yaml:2:# The report ENGINE (config language, renderers) lives in the
shared `obsreport`
README.md:305:> The optional `obsreport` package powers the per-video observability reports; its
tests
README.md:306:> skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-
```

```
obsreport --no-build-isolation`.
run_all_tests.py:17:- The observability **reporting** tests skip cleanly if the optional
``obsreport``
run_all_tests.py:18:  package isn't installed (they use ``pytest.importorskip``). To exercise
them too:
run_all_tests.py:19:      pip install -e ../sports-obsreport --no-build-isolation
run_all_tests.py:43:    obs = "available" if _installed("obsreport") else "NOT installed ?
reporting tests will SKIP"
run_all_tests.py:47:    print(f" obsreport (optional reporting dep): {obs}")
requirements.txt:16:#   obsreport @ git+ssh://git@github.com/avidullu/sports-
obsreport.git@v0.1.3
requirements.txt:18:#   pip install -e ../sports-obsreport --no-build-isolation
docs\COMMERCIALIZATION.md:70:| **`obsreport`** (reports) | permissive (private) | No | ? soft
dep | Pin the `git+ssh@vX` tag once the repo is published (BACKLOG). |

[Showing results with pagination = limit: 30]
```

21. user (2026-06-10T09:29:49.010Z)

```
docs/archives/checkpoints/2026-06-eval-baseline/README.md
import sys
import os
import subprocess
import shutil
import json

def check_command(cmd):
    """Check if a system command is available."""
    return shutil.which(cmd) is not None

def run_pip_install():
    """Install required Python packages."""
    print("Installing Python packages from requirements.txt...")

    # Check if NVIDIA GPU is present
    has_gpu = check_command("nvidia-smi")

    pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r", "requirements.txt"]

    if has_gpu:
        print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
        pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
    else:
        print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
        pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])

    try:
        subprocess.check_call(pip_cmd)
        print("[SUCCESS] Python packages installed.")
        return True
    except subprocess.CalledProcessError as e:
        print(f"[ERROR] Failed to install Python packages: {e}")
        return False

def init_default_config():
    """Initialize a default config.json if it doesn't exist.

    Now delegates to the canonical backend.config loader so that defaults
    stay in sync with the Pydantic models (single source of truth).
    """
    try:
        from backend.config import save_default_config
    except Exception as e:
        print(f"[ERROR] Could not import backend.config loader: {e}")
```

```
        print("[INFO] Falling back to no-op for config initialization.")
        return

    config_path = "config.json"
    if os.path.exists(config_path):
        print(f"[INFO] {config_path} already exists. Skipping initialization.")
        return

    try:
        saved_path = save_default_config(config_path)
        print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
    except Exception as e:
        print(f"[ERROR] Failed to create config.json via loader: {e}")

def run_diagnostics():
    """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
    print("=" * 50)
    print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
    print("=" * 50)

    # 1. Check Python Version
    print(f"Python Version:
{sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
    if sys.version_info.major < 3 or (sys.version_info.major == 3 and sys.version_info.minor <
8):
        print("[WARNING] Python 3.8+ is highly recommended!")
    else:
        print("[PASS] Python version check passed.")

    print("-" * 50)

    # 2. Check FFmpeg / FFprobe
    ffmpeg_ok = check_command("ffmpeg")
    ffprobe_ok = check_command("ffprobe")

    if ffmpeg_ok:
        print("[PASS] FFmpeg is installed and available in PATH.")
    else:
        print("[FAIL] FFmpeg is NOT found in your PATH.")

    if ffprobe_ok:
        print("[PASS] FFprobe is installed and available in PATH.")
    else:
        print("[FAIL] FFprobe is NOT found in your PATH.")

    if not (ffmpeg_ok and ffprobe_ok):
        print("\n" + "!" * 50)
        print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
        print("How to install FFmpeg on Windows:")
        print("  Option 1: Open PowerShell as Administrator and run:")
        print("            winget install Gyan.FFmpeg")
        print("  Option 2: Download the build from https://ffmpeg.org/download.html")
        print("            and add its 'bin' directory to your System PATH environment
variable.")
        print("!" * 50 + "\n")
    else:
        print("[SUCCESS] All essential media command line tools are ready.")

    print("-" * 50)

    # 3. Check ML / GPU (Optional but good)
    try:
        import torch
        cuda_ok = torch.cuda.is_available()
        if cuda_ok:
```

```
            print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
        else:
            print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available. Falling
back to CPU.")
    except ImportError:
        print("[INFO] PyTorch is not installed. Relevance checks will run using lightweight
visual heuristics.")

    print("=" * 50)
    return ffmpeg_ok and ffprobe_ok

if __name__ == "__main__":
    init_default_config()
    media_ok = run_diagnostics()
    run_pip_install()
    print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
```

22. user (2026-06-10T09:29:49.877Z)

```
10:heavy deps (cv2/torch/numpy) stubbed, so they also pass in environments
42:        "for name in ('cv2', 'torch', 'numpy'):\n"
```

23. user (2026-06-10T09:30:04.520Z)

```
"""WASB shuttle-trajectory runner (WSL2 subprocess adapter).

Windows-side counterpart to backend/detectors/wasb_infer.py. It:
  1. stages the video into the WSL filesystem (fast local reads, not /mnt),
  2. copies wasb_infer.py into the WASB repo's src/ dir,
  3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
  4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.

So the only WASB-specific bits live here; the windowing + CSV parsing are shared
with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
"""
import os
import shlex
import logging
import subprocess
from dataclasses import dataclass
from typing import Dict, Any, Optional, Tuple
```

Reuse the model-agnostic helpers ? trajectory shape + windowing are identical. parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on TrackNe

```
from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)
```

Windows path to the inference wrapper we stage into WSL.

```
_INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


@dataclass
class WasbConfig:
    """Resolved settings for a WASB WSL run (built from config.json -> indexing.wasb)."""
    distro: str = "Ubuntu"
    repo_dir: str = "~/models/WASB-SBDT"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "wasb"
    weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
```

```
    sport: str = "badminton"
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
19:**pluggable interfaces**, reusing the registry pattern the codebase already uses for
65:migration ladder, v1), `backend/main.py` (endpoints, sync flow), the registry pattern
100:is delivered through the existing **AI-provider registry with a paid fallback**.
102:## 3. Pluggable abstractions (extend the existing registry pattern)
108:### 3.1 `StorageBackend` registry
126:  byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
142:### 3.2 `ComputeBackend` / `ExecutionBackend` registry
177:are the proven precedent and stay as the **pipeline** layer. The `annotation` registry +
219:**abstract DB access behind a repository layer now** so the engine is swappable, and
235:  writes artifacts + DB rows. **Observability reports** (already per-video) become the
280:The codebase already abstracts the cloud AI behind `provider_registry` /
282:- Every generative capability routes through the **provider registry**.
301:**Data** supplies labels (the `annotation` registry + `candidate_segments` +
377:  backend is swappable ? same ethos as the provider/registry pattern. Don't get locked in.
387:reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
388:tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
392:**(3)** an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
402:1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry +
406:3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness to
407:   **QA quality**; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
413:wrapper; ms-swift is the engine; the **golden QA eval is the report card**.
470:| **P1** | **Identity + multi-tenant data model + upload + highlight persistence + history**
| auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS; `highlights` table;
repository layer; history API + UI |
471:| **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry;
`local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
|
472:| **P3** | **`ComputeBackend` registry + BYO-compute** | `local`/`hosted`/`byo_worker`;
pull-based worker agent; control/data-plane split |
473:| **P4** | **Rich rally/shot semantic metadata & tagging** *(the next value stage after
solid rally detection + highlights)* | shot type (smash / backhand drive / net), rally
**outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ?
query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden
set via the `annotation` registry + `candidate_segments` |
485:focus to **owned-model quality** (P5). The provider registry keeps the paid path one
576:- [`CODE_MAP.md`](CODE_MAP.md) ? current architecture + the registry extension points.
```

24. user (2026-06-10T09:30:13.549Z)

```
380     ingestion + max frames per example; (2) **weight export** + a **commercial DPA**
covering
381     consented, minors-excluded video (biometric, §7); (3) the **inference plan** for the
size
382     you train (a self-managed **7B** is far cheaper to serve than a 30B-A3B MoE).
383
384     ### Training harness ? make it agent-runnable (nanochat-style operability)
385     The goal: **an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled
video
386     and get a trustworthy ship / no-ship verdict, unattended.** Karpathy's **nanochat** is
the
387     reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
388     tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
389     score. It is **not** a usable engine here (text-only, from-scratch GPT-2-class), but its
390     five properties are exactly what make training agent-runnable, and we copy them:
391     **(1)** one command, end-to-end; **(2)** one complexity dial (sane auto-derived
defaults);
392     **(3)** an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
393     **(4)** reproducible + deterministic + cheap; **(5)** minimal/readable.
394
395     **Engine (don't reinvent):** **`ms-swift`** already delivers this for VLMs ? CLI-driven,
396     **native video**, Qwen3-VL support, an eval backend (EvalScope), and reproducibility via
397     `args.json` + `full_determinism`. (LLaMA-Factory is the close second; HF **`nanoVLM`** ?
```

```
398     ~750 lines ? is the minimal reference to *learn* VLM training, not the production path.)
399
400     **The harness = thin glue over components we already have** ? one declarative,
401     agent-invokable job (call it `train_speedrun`):
402     1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry
+
403        `candidate_segments` + golden set ? the labeling seam in `backend/`).
404     2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (§3.2), run as an
orchestration
405        job (§5 / DEFERRED #16): one config, idempotent, logged.
406     3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness
to
407        **QA quality**; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
408     4. **Register** the result as the `owned_vlm` provider (Gemini paid fallback stays)
**only
409        if it beats the bar**.
410
411     So: **golden labels ? ms-swift train (on a compute backend) ? eval gate ? register
412     provider**, behind one command an agent or CI runs. nanochat is the inspiration for the
413     wrapper; ms-swift is the engine; the **golden QA eval is the report card**.
414
415     > **Prerequisite (the crux):** the harness is only as trustworthy as the **golden QA
eval
416     > set**. nanochat's speedrun is meaningful *because* its eval (CORE score) is automatic
and
417     > trustworthy; ours is the golden QA eval ? **build that first**, or an agent-run loop
418     > optimizes blind. (Same "moat = dataset + eval", scaffolding-first principle.)
419
```

25. user (2026-06-10T09:33:03.663Z)

```
Structured output provided successfully
```

26. assistant (2026-06-10T09:33:15.018Z)

```
Structured output delivered. The case for keeping Gen-0/Gen-1 training in the indexer rests on
verified evidence: the authoritative plan docs (`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`,
`docs/PLATFORM_ARCHITECTURE.md` lines 400-413) already design the harness as an extension of
`backend/eval/`; `rally_seq_proto.py` imports four load-bearing indexer modules; the ship-gate
(`backend/eval/regression.py`) requires the indexer's DB and baselines; and the audit's own
drift facts show cross-repo contracts here fork exactly when interfaces churn ? which Gen-0's
interfaces do weekly. Isolation is delivered in-repo via a sibling `training/` package with an
executable one-way import rule, `requirements-training.txt` extras, a third CI job, and an
artifact-only consumption path mirroring the documented DetectorRunner lazy-import rule ? with
an explicit, doc-committed split trigger at Gen-2/ms-swift.
```